German Reader & Vocabulary PWA — Spec

Oct 12, 2026 · @enes

Overview and constraints

Build a private, phone-first PWA for one user (a native Turkish speaker at German B2) that does two things well: daily German reading at B1–C2 and an advanced vocabulary trainer fed by that reading. It is never published, has no other users and must run at zero cost.

Hard constraints

- **Single user, private.** No sign-up, no multi-tenancy, no server. The app code is public; all reading content and personal progress live in a private GitHub repo.
- **Zero extra cost.** Only free tiers and the user's existing Claude subscription. Never enable billing anywhere. If a feature would require payment, drop or redesign it.
- **GitHub only.** Hosting on GitHub Pages, scheduled jobs on GitHub Actions, storage as JSON files in repos. No Cloudflare, Supabase or other backend.
- **Phone-first PWA.** Installable to the home screen on iOS and Android. No App Store, no Apple Developer account, no Mac required. Must also work in a desktop browser.
- **UI language: Turkish.** All explanations, translations and feedback in Turkish. Reading content in German.
- **Developer works on Windows.** All tooling (Node, Git, Python scripts) must run on Windows.

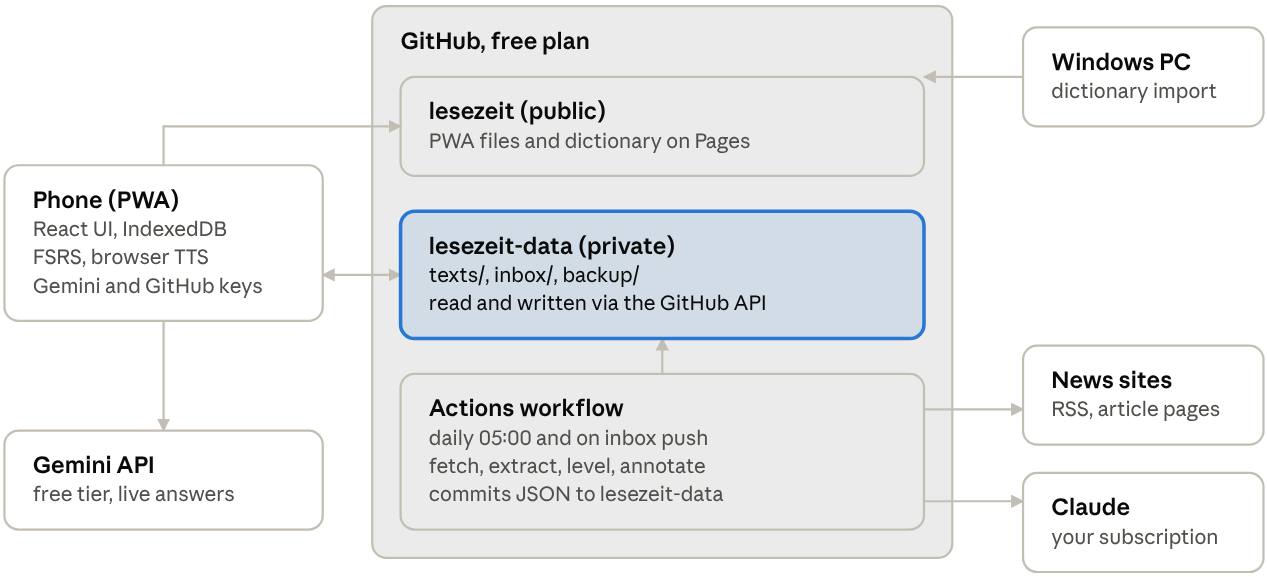
Out of scope: speaking practice, speech recognition, lessons/courses, A1–A2 content, payments, analytics, push notifications (v1).

Reference: recepyagmur/EnglishPractice (MIT) for ideas only: SRS flow, word popup, AI-output verification step, "own sentence" checking. Do not copy its content files.

Architecture

Everything runs on GitHub: a public repo serves the app, a private repo holds the texts and a backup of the user's progress, and a daily GitHub Actions job prepares the texts with Claude. Gemini's free tier answers only the live questions asked on the phone.

Everything runs on GitHub; Claude prepares texts, Gemini answers live



architecture · two GitHub repos, Claude in Actions, Gemini on the phone

Two repos. `lesezeit` (public) holds the app source, the pipeline scripts and the dictionary shards, and deploys the PWA to GitHub Pages. `lesezeit-data` (private) holds `texts/`, `inbox/` and `backup/`, and runs the daily workflow, which checks out the pipeline scripts from the public repo. Full news texts never appear in the public repo, because GitHub Free serves Pages only from public repos.

Keys stay on the phone. On first launch a setup screen asks for a fine-grained GitHub token (only `lesezeit-data`, contents read/write, actions read/write) and a Gemini API key. Both are stored in IndexedDB on the device, never committed. The Claude token lives only as an Actions secret in the private repo.

Layer	Choice	Reason
Frontend	Vite + React + TypeScript, Tailwind, vite-plugin-pwa	Installable PWA, works offline
Hosting	GitHub Pages from `lesezeit` (deployed by an Actions workflow)	Free, public code only
Device storage	IndexedDB via Dexie: cards, word status, logs, settings, keys	No server needed for one user

Layer	Choice	Reason
Content storage	JSON files in private `lesezeit-data`, read through the GitHub REST API with the token	Private, versioned, free
Scheduling	GitHub Actions `schedule` (daily, about 05:00 Europe/Berlin) + `push` on `inbox/`	Free; private repos get 2,000 min/month
Pipeline AI	Claude via the user's subscription (`claude -p` with an OAuth token in Actions)	Best German quality, no extra cost
Live AI on phone	Gemini API free tier, called directly from the browser	Instant answers, no card
Spaced repetition	ts-fsrs, on device	FSRS scheduling
Speech	Web Speech API (speechSynthesis)	On-device German voices, free
Dictionary	Wiktionary via kaikki.org, sharded JSON in the public repo	Free data (CC BY-SA), imported once

Content pipeline

Every morning a GitHub Actions workflow in `lesezeit-data` turns real German news into leveled, fully annotated JSON texts, using Claude through the user's subscription. Everything the reader shows is prepared there once; the phone only reads files.

Sources (config file, easy to extend). Start with free, full-text public outlets: tagesschau.de, deutschlandfunk.de, dw.com/de, zdfheute, br24, mdr.de, swr.de. Zeit, Spiegel and SZ only for items without a paywall flag. Verify each feed URL during setup; drop feeds that only return teasers and cannot be extracted.

Daily workflow (`schedule`, about 05:00 Europe/Berlin; GitHub may start it late). It first checks out `lesezeit-data` plus the pipeline scripts from the public repo and reads `backup/progress.json` (known lemmas, due cards) written by the phone.

1. Fetch all feeds, dedupe by URL hash, skip items already stored.

2. Fetch the article page and extract the main text. Use Node with @mozilla/readability and jsdom, with per-source selectors as fallback. Reject texts under 250 words, paywalled texts and live blogs.
3. Store title, source, URL, date, plain text and word count. Keep the URL for an "Original öffnen" link.
4. **Level it (B1, B2, C1, C2).** Compute heuristics: average sentence length, share of words outside the 5,000 most frequent lemmas, subordinate clauses, passive and Konjunktiv counts. Then ask Claude for `{level, confidence, reasons_tr}`. If AI and heuristics differ by two levels or more, keep the AI level but flag the text.
5. **Annotate it** with Claude, for every stored text. Split into sentences; send batches of about 8 sentences and get back structured JSON per token: lemma, POS, gender and plural for nouns, case for each noun phrase, separable-verb links ("fängt … an" → anfangen), reflexive use, multi-word expressions (Nomen-Verb-Verbindungen, verb + preposition, idioms), a Turkish translation per sentence and short Turkish grammar notes. Cross-check every lemma against the dictionary; mismatches are flagged, not silently trusted.
6. **Generate comprehension material:** 5 questions (multiple choice, richtig/falsch/nicht im Text, one open question), each with the answer and the sentence that proves it.
7. **Pick the day's texts:** 2 per level by default (setting), preferring varied topics and texts whose known-word coverage is 90–98% for this user (from the progress backup).

Output: each text becomes `texts/YYYY-MM-DD/<id>.json` (sentences, tokens, questions), `texts/index.json` is updated, and the workflow commits and pushes.

Generated texts (on demand)

- The phone writes a request file to `inbox/` through the GitHub API; a `push` on `inbox/**` starts the same workflow, which handles it and deletes the request. The text appears after a few minutes and shows as "hazırlanıyor" until then.
- Inputs: level, topic (free text or a picked category), length (150 / 300 / 600 words), genre (news, opinion, story, formal letter / Behördendeutsch, exam style like Goethe or TestDaF), and up to 8 target lemmas.
- Claude writes the text, then a separate validation call checks grammar, level fit and that each target word is used naturally in the intended sense. Failed validation → regenerate at most twice, then keep the best attempt with a warning.
- Labeled "KI-Text" in the UI and annotated through steps 5–6.
- **Context review:** the daily run also writes one short text built from the cards that are due in the backup, so review happens inside reading.

User import: paste German text or a URL in the app; it goes to `inbox/` and is processed like a news item (steps 2–6).

Reader

The reader shows German text with every word tappable, colors words by how well the user knows them, and ends each text with comprehension work.

Home ("Heute")

- Today's texts grouped by level, each with title, source, word count, estimated minutes and known-word coverage %.
- Level filter (B1–C2), topic filter, "Neuer KI-Text" button, archive of past texts.

Word coloring (per lemma, not per form)

- New (never seen): no highlight but underlined dotted. Learning (has a card): yellow background. Known: plain. Ignored (names, numbers): plain.
- Nouns optionally show article color on tap: der = blue, die = red, das = green.
- On finishing a text: "Rest als bekannt markieren" marks all remaining new lemmas as known (LingQ-style).

Tap a word → bottom sheet

- Surface form and lemma; for separable verbs the full verb ("an … fangen" → anfangen) with both parts highlighted in the text.
- Nouns: article + plural + genitive. Verbs: Präteritum, Partizip II, auxiliary (haben/sein), case it governs, reflexive. Adjectives: comparative.
- Case and role in **this** sentence ("Dativ, weil: mit").
- Turkish meaning **in this context** first, then other dictionary senses.
- Part of a multi-word expression? Show the whole expression and its meaning.
- Buttons: pronounce, add to vocabulary (with this sentence as context), mark known, ignore.

Tap a sentence (long-press or sentence button)

- Turkish translation, structure breakdown (main clause / Nebensatz, verb position, passive, Konjunktiv II, Partizip-Attribute), pronounce sentence.
- "Neden böyle?" opens a free-form Turkish question to the AI about that sentence.

Audio: browser speechSynthesis with a de-DE voice, sentence-by-sentence with the current sentence highlighted, speed 0.7×–1.2×.

After reading

1. Comprehension questions with instant check and the proving sentence highlighted.
2. Optional summary: user writes 3–5 German sentences; AI returns errors marked red/green with Turkish explanations and a corrected version. Errors use a fixed category list (Artikel, Kasus, Verbstellung, Konjugation, Adjektivendung, Präposition, Rechtschreibung, Wortwahl, Sonstiges).
3. Stats saved: reading time, new words added, score.

Guess mode (setting, default on): tapping a new or learning word first shows only the word inside its sentence and asks the user to guess the meaning. "Anlamı göster" reveals the full word sheet with "Tahminin doğru muydu?" (Evet / Hayır). Hayır adds the word to the cards with this sentence as context; known words skip guess mode.

Display settings: font size, line height, serif/sans, dark mode, "translation hidden until tapped".

Vocabulary trainer

Words and expressions saved while reading become FSRS cards, reviewed through a mix of exercise types chosen by each card's state. Scheduling uses the `ts-fsrs` library (FSRS, the algorithm Anki now uses), not SM-2.

Cards

- A card is a lemma **or** a multi-word expression ("eine Entscheidung treffen", "sich verlassen auf").
- Each card keeps every context sentence it was saved from (with a link back to the text), its Turkish meaning in that context, and the grammar data from the dictionary.
- One FSRS state per card. Exercises map their result to Again / Hard / Good / Easy (wrong = Again, slow or hinted = Hard, correct = Good, correct and fast on a mature card = Easy).

Exercise types

Exercise	What it trains	How it works
Flashcard	Recognition and recall	DE → TR or TR → DE, flip, self-rate
Matching (Eşleştirme)	Fast association	5–6 pairs, tap to match, timed. Modes: DE ↔ TR, noun ↔ verb (Entscheidung ↔ treffen), verb ↔ preposition (sich verlassen ↔ auf), word ↔ German definition, synonym ↔ synonym
Artikel & Plural	Noun grammar	Pick der/die/das, then type the plural
Cloze in context	Meaning in use	Original reading sentence with the word blanked; type it (lemma hint optional)
Listen & type	Spelling and listening	TTS says the word or sentence, user types it
Production	Active recall	Turkish meaning shown, type the German word with article
Verb forms & Rektion	Verb grammar	Präteritum, Partizip II, haben/sein, governed case or preposition
Word family	Derivation	Build related forms (entscheiden → Entscheidung → entschieden)
Synonym / antonym	Nuance	Multiple choice from dictionary relations
Own sentence	Real use	Write a sentence with the word; AI checks grammar and whether the sense matches, feedback in Turkish

Word family shows a base word and asks for one derived form (4 options, distractors are invented forms with wrong suffixes). After answering, the whole family is shown with Turkish meanings and the suffix rule (e.g. "-ung → always die").

Audio in exercises: tapping any German item plays its pronunciation (matching: every German tile on select; artikel: the noun with its article after answering; cloze: the full sentence once the gap is filled). Turkish items are never spoken. One sound toggle in settings turns all of it off.

Session composer

- New cards: flashcard → matching → cloze. Young cards: cloze, listen & type, artikel, matching. Mature cards: production, own sentence, verb forms, word family.
- Daily limits (settings): new cards per day (default 15), max reviews (default 150). Wrong cards repeat once at the end of the session.
- Session length options: 5, 10, 20 minutes, or "all due".

Extra modes

- **Context review:** read the daily generated text that contains due cards; tapping a due word counts as a failed review, reading past it counts as a pass.
- **Expression decks:** separate filters for Nomen-Verb-Verbindungen, verbs with prepositions and idioms.
- **Leech handling:** a card failed 6 times is flagged and shown with an AI mnemonic and extra example sentences.

Stats: cards by state, reviews per day, retention, streak, words read per week.

Card list: each card shows its most recent context sentence with the word highlighted; tapping it opens that text scrolled to the sentence, which is highlighted.

Weekly progress (low priority): on the home screen, a small row of reading minutes per day for the current week, plus texts read, new cards and average known-word coverage on texts at the user's main level.

Data model

Data lives in three places: dictionary shards in the public repo, texts and the progress backup as JSON in the private repo, and the user's working state in IndexedDB on the phone. Validate every JSON file with a shared Zod schema in both the pipeline and the app.

Where	File or store	Contents
`lesezeit` (public)	`public/dict/forms/<xx>.json`	Inflected form → lemma ids, sharded by the first two letters

Where	File or store	Contents
`lesezeit` (public)	`public/dict/lemmas/<xx>.json`	Lemma, POS, gender, plural, genitive, Präteritum, Partizip II, auxiliary, Rektion, separable prefix, frequency rank, English gloss, synonyms, antonyms, family
`lesezeit-data` (private)	`texts/index.json`	Every text: id, date, kind (news / generated / imported / review), level, flagged, title, source, URL, word count
`lesezeit-data` (private)	`texts/YYYY-MM-DD/<id>.json`	Body, sentences (translation_tr, grammar_notes_tr), tokens (surface, lemma, POS, case, sep link, mwe id, meaning_tr), MWEs, questions with proof sentence
`lesezeit-data` (private)	`inbox/<timestamp>.json`	Requests from the phone: generate or import
`lesezeit-data` (private)	`backup/progress.json`	Latest snapshot of all IndexedDB stores, written by the phone
`lesezeit-data` (private)	`sources.json`	Feed list: name, feed URL, selector hints, enabled
Phone, IndexedDB	`wordStatus`	lemma or mwe id → new / learning / known / ignored
Phone, IndexedDB	`cards`	id, lemma or mwe id, meaning_tr, context sentence refs, FSRS state, due, leech count
Phone, IndexedDB	`reviewLog`, `readingLog`, `writingAttempts`	History for stats and FSRS
Phone, IndexedDB	`textCache`	Downloaded texts for offline reading

Where	File or store	Contents
Phone, IndexedDB	`aiCache`	Gemini answers keyed by input hash
Phone, IndexedDB	`settings`	Limits, levels, font, voice, sound, guess mode, GitHub token, Gemini key

Dictionary import: a script run once on the Windows PC downloads the kaikki.org German JSONL, keeps only the needed fields, adds a frequency rank from a free German frequency list, and writes the shards into the public repo. Keep each shard small (under about 300 KB) so a tap loads fast on mobile data, and add an attribution page for Wiktionary (CC BY-SA).

Backup and sync: the app writes `backup/progress.json` after each session (debounced, at most every 10 minutes) and has an "Yedekten geri yükle" action for a new phone. The last write wins; a copy of the previous backup is kept as `backup/progress.prev.json`.

AI usage, limits and caching

Claude does all the heavy, quality-critical work in the pipeline through the user's subscription; Gemini's free tier handles only the short, interactive answers on the phone. Neither is ever billed.

- **Claude in Actions.** Install Claude Code in the workflow and call `claude -p` with `--output-format json`, authenticated by a long-lived OAuth token from `claude setup-token`, stored as the Actions secret `CLAUDE_CODE_OAUTH_TOKEN`. Verify these commands against the current Claude Code docs at setup. This draws on the subscription's normal usage limits, shared with chat and Claude Code.
- **Limit handling.** If Claude reports a usage limit, the workflow commits what it finished, leaves the rest in a `pending` list and continues on the next run. Never fall back to a paid API key.
- **Pipeline budget.** Setting `maxTextsPerRun` (default 8) plus a cap on annotation batches per run, so a normal day uses a small, predictable share of the limit.
- **Gemini on the phone.** Used only for: checking the user's own sentence, summary feedback, the "Neden böyle?" question, and meanings for words the pipeline missed. Called directly from the browser with the key from settings. Model name is a setting; at setup pick one marked free of charge on the Gemini pricing page. Never enable billing.

- **Structured output.** Both providers return JSON validated with Zod. Invalid JSON → one retry, then mark the item as failed.
- **Caching.** The pipeline never re-annotates a stored text. On the phone, Gemini answers are cached in IndexedDB by input hash; when the free quota is used up (HTTP 429), show a short Turkish notice and fall back to dictionary data.
- **Verification.** Generated texts and comprehension questions pass an independent Claude check before they are written. Lemmas are checked against the dictionary.
- **Prompts live in versioned files** (`/prompts/*.md`) with a few-shot example each; Turkish for explanations, German for content.
- **Privacy note:** Gemini free-tier inputs may be used by Google to improve its products. Only the user's practice sentences and short text excerpts are sent; no personal data.

Milestones

Build in this order; each milestone ends with something usable on the phone and its acceptance check passing.

1. **Setup.** Create `lesezeit` (public) and `lesezeit-data` (private). Vite + React + TS PWA, Pages deploy workflow, setup screen for the GitHub token and Gemini key. *Done when:* the app installs to the iPhone home screen and shows the contents of a test file read from `lesezeit-data`.
2. **Dictionary.** Import script, shards, lookup in the app (form → lemma → grammar data). *Done when:* "fing", "Häusern" and "angefangen" each resolve to the right lemma with article, plural or verb forms.
3. **Pipeline without AI.** Workflow, feeds, extraction, heuristics, JSON output, commit. *Done when:* a manual run commits at least 8 clean texts, and no teaser-only or paywalled text gets through.
4. **Claude in the pipeline.** OAuth token secret, leveling, full annotation, questions, limit handling. *Done when:* a run produces annotated texts and a forced limit error leaves a correct `pending` list.
5. **Reader.** Home list, reading view, word sheet from annotations + dictionary, coloring, guess mode, sentence sheet, TTS, questions, offline cache. *Done when:* tapping "an" in "Er fängt morgen an" opens anfangen, and a downloaded text opens in airplane mode.
6. **Vocabulary core.** Cards with context sentences, FSRS, flashcards, matching (all modes, with audio), artikel & plural, cloze, word family. *Done when:* a 10-minute mixed session runs and due dates update correctly.

7. **Backup and coverage.** Progress backup, restore, coverage-based picks, context-review text. *Done when:* a word marked known on the phone lowers its weight in the next day's picks.
8. **Writing and live AI.** Own sentence, summary feedback, "Neden böyle?" via Gemini. *Done when:* a wrong summary sentence comes back with the error marked and explained in Turkish.
9. **Generated and imported texts.** `inbox/` requests, generator + validator. *Done when:* a requested B2 text with 5 target words appears within 10 minutes.
10. **Polish.** Stats, weekly progress, settings, dark mode.